# Module 12: Building the Organization Layer

**Modules, Packages, and Project Structure**

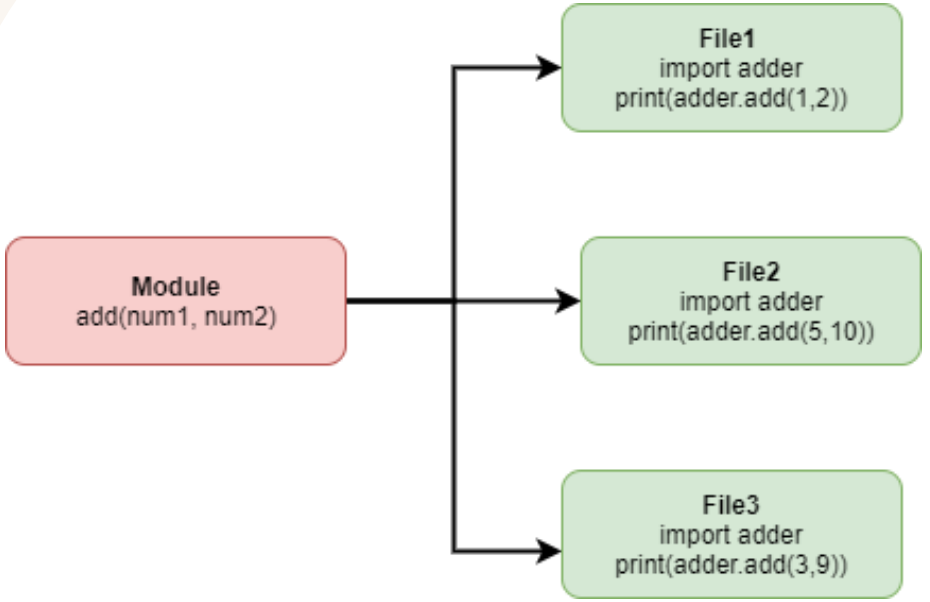
Until now, all our code has lived in one place. But real applications have thousands of lines of code. Today, we learn how to break that code into logical pieces so it stays manageable and professional.

Presented By: Zaryab Ahmad

# What is a Module?

**The Single File Logic**

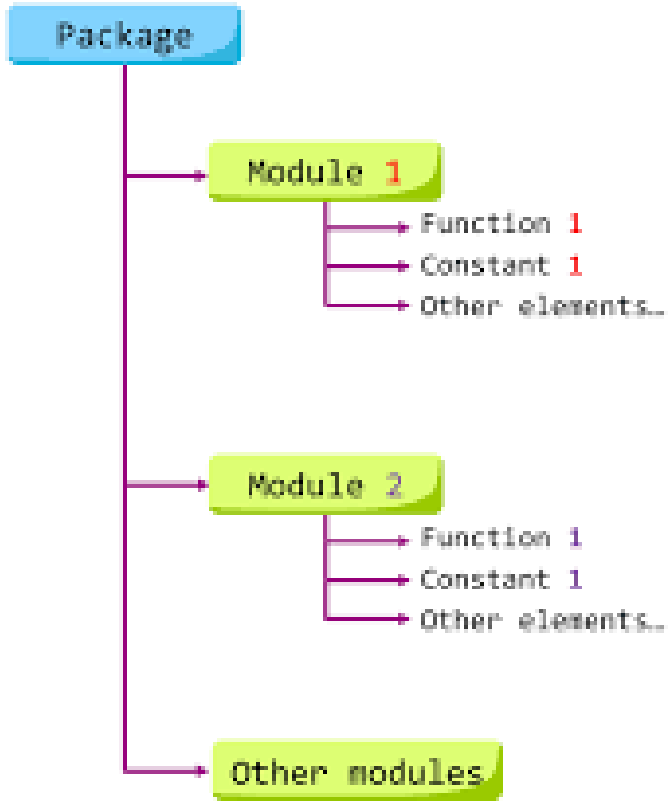
- A Module is simply a *.py* file containing functions, classes, and variables.
- Modularity: It allows you to separate concerns (e.g., one file for database logic, one for calculations).
- Imports: Use import *module_name* to access its contents in another file.



# What is a Package?

**The Folder Hierarchy**

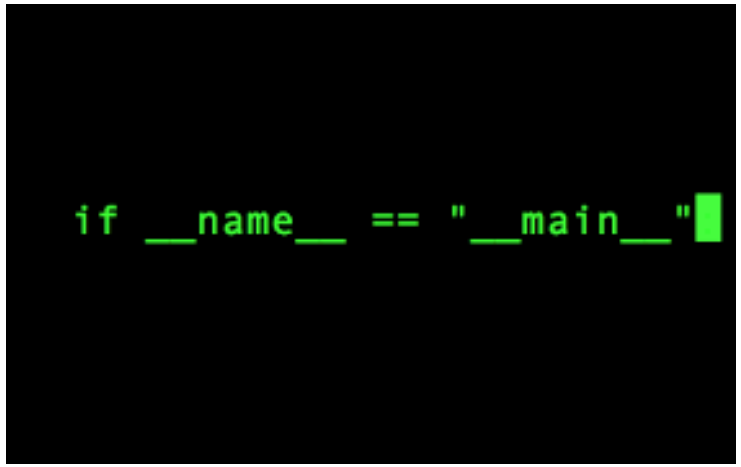
- A Package is a directory containing multiple modules.
- *__init__.py*: A special file that tells Python, 'This folder is a package.
- 'Namespacing: Prevents naming conflicts (e.g., *math_utils.add* vs *string_utils.add*).

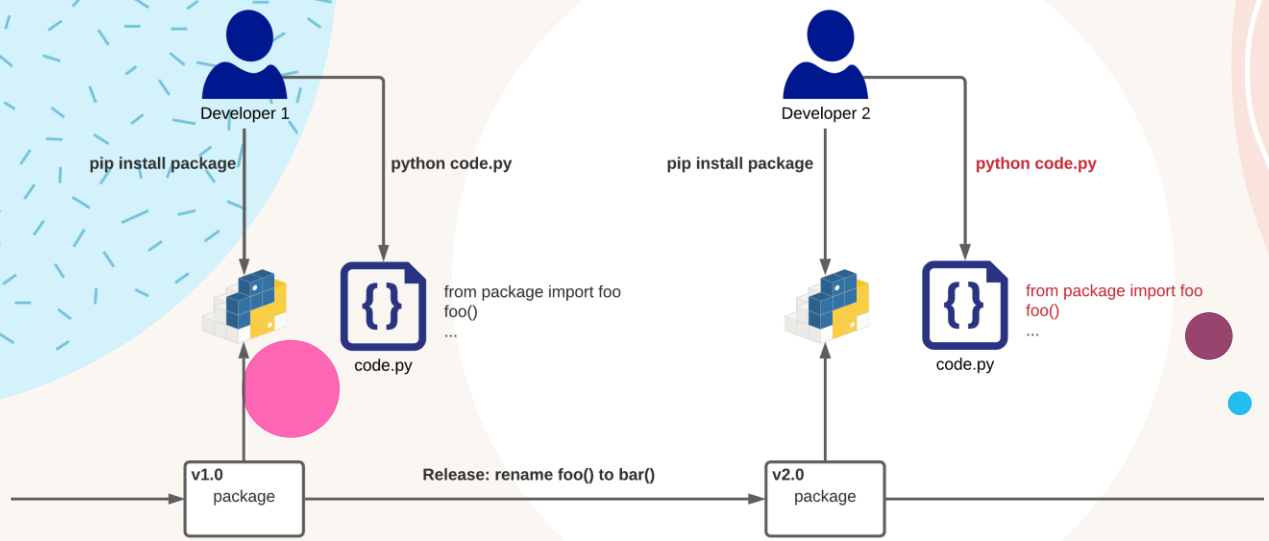


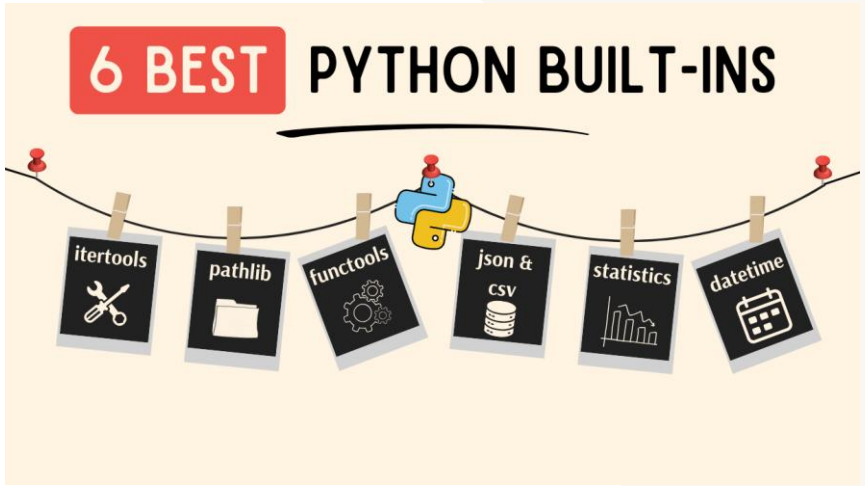
# The "Main" Guard

$$if\ \_name\_ == "\_main\_":$$

- Determines if a script is being run directly or imported as a library.
- Prevents code from running automatically when you only want to import a function.






# The Standard Library & Pip

**Standing on the Shoulders of Giants**

- Standard Library: Batteries included (e.g., *math*, *os*, *sys*, *datetime*, *itertools*).
- Pip: The Python Package Index. How we install external tools.
- *requirements.txt*: The shopping list for your project's environment.